

# Temperature Control System with Web Visualization

Kirthika Gopalan  
427320  
Automation & Control

September 2025

## Contents

|          |                          |          |
|----------|--------------------------|----------|
| <b>1</b> | <b>Introduction</b>      | <b>2</b> |
| <b>2</b> | <b>Hardware used</b>     | <b>2</b> |
| <b>3</b> | <b>DS18B20</b>           | <b>2</b> |
| 3.1      | Introduction . . . . .   | 2        |
| 3.2      | Wiring . . . . .         | 3        |
| 3.3      | Coding . . . . .         | 3        |
|          | 3.3.1 temp.py . . . . .  | 4        |
|          | 3.3.2 web.html . . . . . | 5        |
| 3.4      | Result . . . . .         | 5        |

# 1 Introduction

The goal of this project is to implement a web visualization of temperature using DS18B20 sensor. This report contains the implementation details and the project results.

## 2 Hardware used

For this project, the hardware setup contains:

- Raspberry Pi 5
- Temperature Sensor DS18B20

## 3 DS18B20

### 3.1 Introduction

The DS18B20 is a digital temperature sensor commonly used to measure temperature. It operates using the 1-Wire communication protocol.



Figure 1: DS18B20-Temperature Sensor

Here is an explanation of the three wires:

- Red: Power Supply 3.3V.

- Yellow: Data pin (communication with Raspberry Pi).
- Black: Ground.

### 3.2 Wiring

From the figure 2, DS18B20 sensor is connected with the Raspberry Pi in a standard way for VCC and Ground. The data pin of the sensor requires a pull-up resistor of 4.7 kOhm to 10 kOhm. For this, the GPIO pin 4 is chosen which internally contains a pull-up resistor. The internal 1-Wire configuration of Raspberry Pi should be enabled in order to detect the sensor. [3]

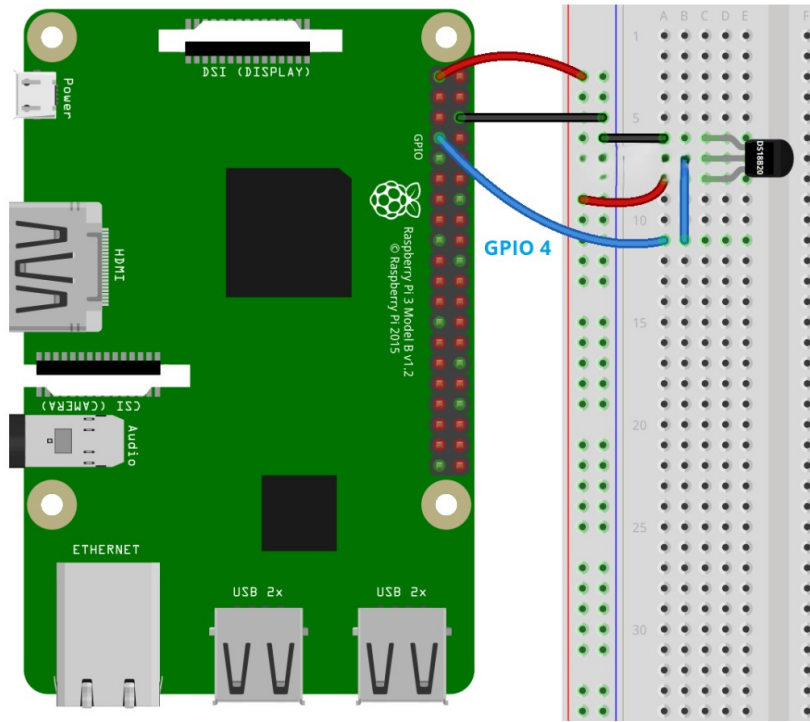


Figure 2: Wire Connection

### 3.3 Coding

The project source file can be found in `/temp.py` and `/templates/web.html` in the home folder. The files are also attached with the report in the Zip file. In the following sections, you can find the details of the codes.

### 3.3.1 temp.py

The python file that reads the temperature data from a DS18B20 temperature sensor(using 1-wire protocol) and displays in the web page using the flash library in the local server under port 5000. [2] //

```
app = Flask(__name__)

SENSOR_PATH = "/sys/bus/w1/devices/28-01204b4064b4/w1_slave"

def read_temp():
    try:
        with open(SENSOR_PATH, 'r') as f:
            lines = f.readlines()
            if "YES" in lines[0]:
                temp_str = lines[1].split("t=")[-1]
                temp_c = float(temp_str) / 1000.0
                return temp_c
            else:
                return None
    except FileNotFoundError:
        return None
```

Figure 3: Read DS18B20 temperature sensor

```
@app.route('/')
def index():
    # Web Visualization
    temp = read_temp()
    if temp is not None:
        temperature = f"{temp:.2f} °C"
    else:
        temperature = "Unable to read temperature"
    return render_template('web.html', temperature=temperature)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Figure 4: Write temperature sensor output to the web variable

### 3.3.2 web.html

This html file contains the template for rendering the current temperature from the Flask application. [1]

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Current Temperature</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
    }
    .temperature {
      font-size: 48px;
      color: #333;
    }
    .status {
      font-size: 18px;
      color: gray;
    }
  </style>
</head>
<body>
  <h1>Current Temperature</h1>
  <div class="temperature">{{ temperature }}</div>
</body>
</html>
```

Figure 5: HTML Template for rendering the current temperature value

## 3.4 Result

Once the hardware connection is verified, the python file temp.py is executed. The final temperature is sent to the local IP which can be reached by other locally connected devices Eg. laptops or mobile phones. The Figure 6 and Figure 7 shows the temperature before and after warm condition.

16:49



**Current Temperature**

23.44°C

16:50



**Current Temperature**

25.25°C

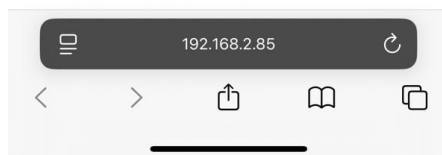


Figure 6: Room temperature

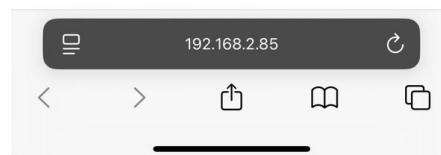


Figure 7: Temperature under warm condition

## References

- [1] Flask Documentation(3.1.x). Flask documentation tutorials. URL: <https://flask.palletsprojects.com/en/stable/tutorial/templates/>.
- [2] Robert Elder. Ds18b20 setup documentation, 2019. URL: <https://blog.robertelder.org/ds18b20-raspberry-pi-setup-pullup/>.
- [3] Raspberry Pi Foundation. Raspberry pi documentation, 2025. URL: <https://www.raspberrypi.com/documentation/>.